# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## ASSESSMENT TOOL 3 – Comparative Analysis

| | | | |
|---|---|---|---|
| **Course Code:** | CSA0305 | **Course Title:** | Data Structures |
| **CO Assessed:** | CO4 – Articulation (BL3) | **Assessment:** | Assessment Tool 3 – Comparative Analysis |
| **Weightage:** | 30% of CIA | **Total Marks:** | 100 |
| **CO4 Statement:** | *Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.* | | |
| **Student Name:** | Shahana | **Reg. No.:** | 192524479 |
| **Section / Batch:** | Slot A | **Date:** | 26/08/2026 |

**Code of Conduct**

I Shahana (192524479) *(Name / Reg No)*

*certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Shahana

**Instructions**

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

| Section / Topic | Focus | Q Nos | Marks |
|---|---|---|---|
| Hashing | Linear Probing & Separate Chaining | 1 | 20 |
| Sorting | Merge Sort & Quick Sort | 2 | 20 |
| Hashing | Quadratic Probing and Double Hashing | 3 | 20 |
| Sorting Algorithms | Complexity Analysis | 4 | 20 |
| Quick Sort | Recursive and Iterative Implementations | 5 | 20 |
| | **GRAND TOTAL:** | | **100 Marks** |

## Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
   a) Clustering effect
   b) Memory usage
   c) Performance under high load factor
   d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
   a) Clustering issues
   b) Collision resolution efficiency
   c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
   a) Practical significance
   b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
   a) Space complexity (stack usage)
   b) Ease of implementation
   c) Risk of stack overflow

**Rubrics for Calculation**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Concept Understanding | 20 | Demonstrates complete and accurate understanding of all data structures with advanced insights | Shows clear understanding with minor gaps | Basic understanding; some misconceptions present | Limited or incorrect understanding |
| Comparative Analysis Depth | 20 | Thorough, multi-dimensional comparison with strong justification and critical evaluation | Good comparison with relevant factors considered | Limited comparison; lacks depth or justification | Minimal or incorrect comparison |
| Use of Parameters (Time/Space/Operations) | 30 | All parameters analysed accurately with complexity discussion | Most parameters covered correctly | Few parameters considered; some inaccuracies | Parameters missing or incorrect |
| Critical Thinking | 20 | Strong justification of why one structure is better in given contexts | Some justification present | Limited reasoning | No critical reasoning |
| Clarity | 10 | Exceptionally well-structured, logical flow, clear tables/diagrams | Well-organized with minor issues | Some organization issues; lacks clarity | Poor structure and readability |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# CO4 - AT3 : Comparitive Analysis

Name: Shahana
Reg.No: 192524479
Course: CSA0305 - Data Structures.

1. Compare Linear Probing and Separate Chaining

| Basis | Linear Probing | Separate Chaining |
|---|---|---|
| Clustering | Primary clustering - once a cluster forms, it grows faster since every new key hashing nearby must probe past it. | No clustering - each bucket is independent, collisions just extend that bucket's list. |
| Memory usage | No pointer overhead, but table must always keep empty slots (load factor $<1$) or insertion fails. | Pointer overhead per node, but load factor can exceed 1 without breaking. |
| High load factor performance | Degrades sharply - probe count approaches $O(n)$ as $\alpha$ nears 1 | Degrades gracefully - average search stays $O(1+\alpha)$ |
| Deletion | Hard - needs tombstones to avoid breaking probe sequences | Easy - just unlink the node from the list. |
| Large-scale suitability | Poor - hard load factor ceiling limits growth | Good - predictable degradation, no hard cap. |

2. Compare Quick Sort and Merge Sort.

| Basis | Quick Sort | Merge Sort |
|---|---|---|
| Memory (on a list) | Partitions via pointer re-linking, no new nodes but recursion stack can hit O(n) with bad pivots. | Merging is pure pointer re-linking, O(log n) stack, no auxillary array needed at all. |
| Cache behaviour | Poor - scattered list nodes kill the sequential access advantage quicksort relies on for arrays | Poor too, but performance was never dependent on locality to begin with. |
| Time complexity | O(n log n) average, but O(n2) worst case and good pivots (median-of-three) need random access, which lists lack. | O(n log n) guaranteed in best, average and worst cases. |
| Random access needed | Ideally yes, for good pivot selection | No - works purely off sequential pointers. |
| Better fit | Arrays | Linked Lists. |

Compare Quadratic Probing and Double Hashing

| Basis | Quadratic Probing | Double Hashing |
|---|---|---|
| Clustering Issues | Removes primary clustering, but secondary clustering remains - same initial slot means identical probe sequence. | Removes both primary and secondary clustering - probe sequence depends on a second hash, unique per key. |
| Collision resolution efficiency | Works well at $\alpha <= 0.5$ with prime table size, but may skip over empty slots that do exist. | Approximate uniform hashing, reliably finds empty slots even at higher load. |
| Suitability for dense tables | Not ideal - coverage gaps appear as table fills. | Well suited - uniform probe distribution holds even as $\alpha \to 1$ |

4. Compare best and worst-case time complexity.

| Algorithm | Best Case | Worst Case |
|---|---|---|
| Bubble sort | O(n) - already sorted | O(n2) |
| Insertion sort | O(n) - already sorted | O(n2) |
| Selection sort | O(n2) - no short-cut even if sorted | O(n2) |
| Quick sort | O(n log n) - balanced partitions | O(n2) - poor pivot choice |
| Merge sort | O(n log n) | O(n log n) - same in all cases |
| Heap sort | O(n log n) | O(n log n) - same in all cases. |

5. Compare Recursive and Iterative quick-sort

| Basis | Recursive | Iterative |
|---|---|---|
| Space complexity | Implicit call stack, O(log n) average but O(n) worst case on unbalanced partitions | Explicit stack - same bounds but O(log n) worst case achievable if smaller partition is always processed first. |
| Ease of implementation | Simple, intuitive, mirrors divide and conquer directly | More complex - manual push/pop of (low, high) index pairs, easier to introduce bugs |
| Risk of stack overflow | Higher - deep recursion on adversarial/sorted input can exhaust call stack | Lower - heap allocated explicit stack, bounded at O(log n) with smaller-partition-first trick |